Conference Paper Review System

Mid-sem: EGC 211P Section A

Duration: 90 minutes

Practice reconstruction of the 2025 Mid-sem, formatted to the 2026 Mid-sem guidelines

READ THE INSTRUCTIONS CAREFULLY

Contents of Midsem.zip

```
question.pdf                                          ← this problem statement
Sample_Runs.pdf                                ← end-to-end traces of the driver
skeleton.cpp                                            ← complete, then submit
local_tester/                                    ← local testcases (do not edit)
```

Write every class yourself in `skeleton.cpp`. It contains only the `PaperStatus` enum, its display strings and an empty `main`. Nothing else is given: translate the UML (Section 3) into C++ and implement the behaviour described in Sections 4–7.

Write the classes in the order of the sections: `Paper` → `ConferencePaper` → `Reviewer` → `Conference`. Each class depends on the previous ones.

Names, parameter types and order, return types and `const` must match the UML exactly. The testcases call your methods directly; a different signature is a compile failure for that testcase.

Compilation error ⇒ 0 marks. During the exam DOMjudge checks only compilation. If some code does not compile, comment it out before submitting.
DO NOT use `bits/stdc++.h`. If used, 50% of the earned points are deducted.

Copy-paste output formats from this pdf; do not type them.

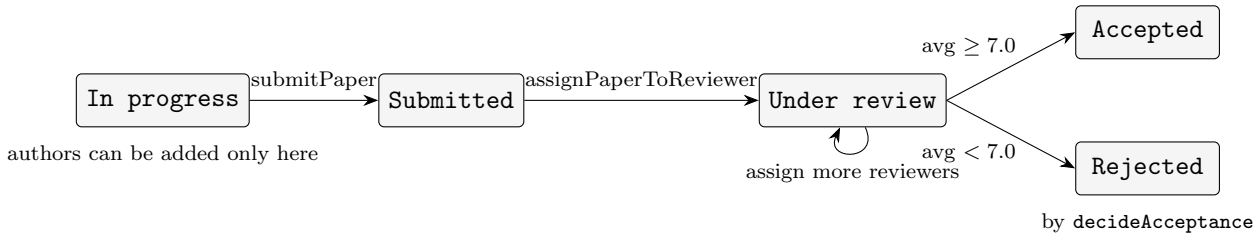
Command to run local testcases (in the folder that has `skeleton.cpp`):
`sh ./local_tester/local_test.sh`

Stop adding new code 5–10 minutes before the end.

1 Application Overview

The application models the review workflow of an academic conference. Authors write a *paper* and, while it is still a draft, may add co-authors. When the draft is ready, it is *submitted* to the conference. The conference then assigns the paper to one or more *reviewers*. A reviewer may never review a paper they co-authored (conflict of interest). Each assigned reviewer gives the paper an integer score from 1 to 10. Finally, the conference decides acceptance: a paper whose average score is at least 7.0 is accepted, otherwise it is rejected.

Every paper moves through the following states. The label on each arrow is the only method allowed to make that change.



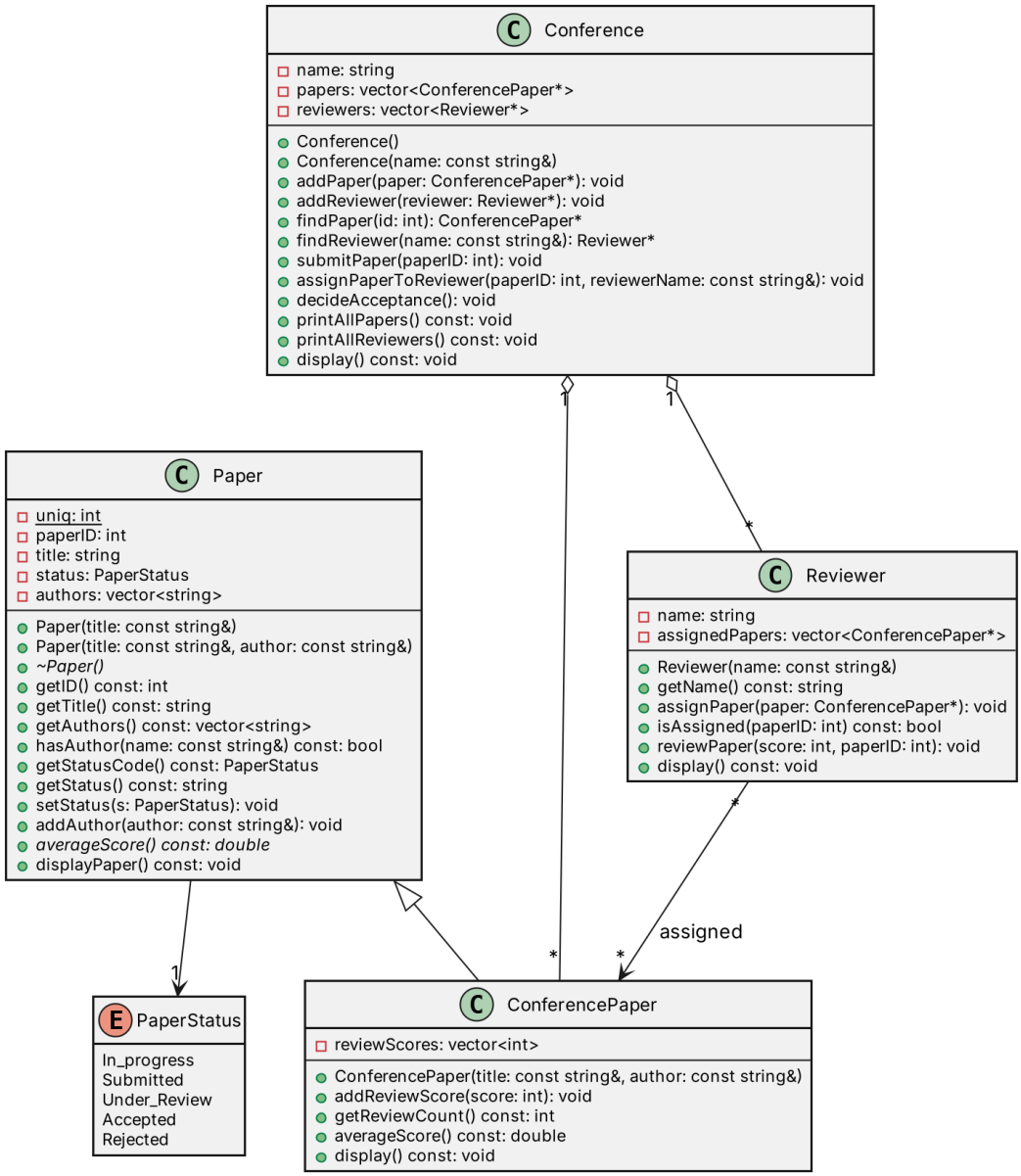
2 Classes

The design has four classes. Implement them in this order:

1. `Paper`: a paper with a unique ID, a title, a status and a list of authors (Section 4).
2. `ConferencePaper`: a `Paper` that also stores its review scores (Section 5).
3. `Reviewer`: a reviewer and the papers assigned to them (Section 6).
4. `Conference`: owns the lists of papers and reviewers and runs the workflow (Section 7).

Add any extra private helpers you need. Do not remove or rename anything in the UML.

3 UML Class Diagram



Notation.

red square / green circle	private / public member
underlined	static member
italic	virtual method (`~Paper` is a virtual destructor)
hollow triangle	inheritance: `ConferencePaper` is a public subclass of `Paper`
hollow diamond	aggregation: `Conference` holds pointers to papers and reviewers it does not create
plain arrow	association: a `Reviewer` points to the papers assigned to them
`1`, `*`	multiplicities
`name: type`, `f(p: T): R`	attribute name and type; method with parameters and return type

Indicative marks. UML translation (all classes and signatures compile against the tester): 20. `Paper`: 15. `ConferencePaper`: 10. `Reviewer`: 20. `Conference`: 35.

4 Paper Class

4.1 IDs and constructors

uniq is a private static counter, initialised to 0 outside the class. Every constructor sets paperID = ++uniq, so the first paper ever created gets ID 1, the next one ID 2, and so on. Every new paper has status In_progress. The two-argument constructor also stores author as the first author. The destructor is virtual and does nothing.

4.2 Getters and status

getID, getTitle and getAuthors return the corresponding member (getAuthors returns a copy, in insertion order). getStatusCode returns the enum value. getStatus returns the display string from the given table, i.e. paperStatus[static_cast<int>(status)]. setStatus overwrites the status. hasAuthor returns true if name is in the author list.

4.3 addAuthor method

Appends author to the author list. If the author is already in the list, nothing happens.
Throw Exceptions:

Case	Throw
Paper status is not In_progress (submitted papers are frozen)	throw paperID; (an int)

4.4 averageScore method (virtual)

A plain Paper has no reviews: return -1.0. ConferencePaper overrides this.

4.5 displayPaper method

Prints the ID, a space, the title, a colon and a space, then every author followed by one space. No newline.

```
<paperID> <title>: <author1> <author2> ...
```

Example, paper 1 titled GraphNets with authors Alice and Carol prints "1 GraphNets: Alice Carol " (note the trailing space).

5 ConferencePaper Class

Public subclass of Paper. The constructor passes title and author to the Paper constructor.

- addReviewScore(score) appends score to reviewScores. No validation here; the reviewer validates.
- getReviewCount() returns the number of stored scores.
- averageScore() overrides the base version: the arithmetic mean of reviewScores as a double, or 0.0 if there are no scores. Example: scores 8 and 7 give 7.5.
- display() calls displayPaper(), then prints the average with exactly two decimals, a space, the status string and a newline:

```
<paperID> <title>: <author1> ... <avg:2dp> <status>\n
```

Example: 1 GraphNets: Alice Carol 7.50 Accepted. Use fixed and setprecision(2) from <iomanip>.

6 `Reviewer` Class

`assignedPapers` stores *pointers* to the conference's paper objects, so a score added through a reviewer changes the actual paper.

6.1 `assignPaper` and `isAssigned`

`assignPaper(paper)` appends the pointer, unless a paper with the same ID is already assigned to this reviewer (then nothing happens). `isAssigned(paperID)` returns whether a paper with that ID is in `assignedPapers`.

6.2 `reviewPaper` method

Read the parameter order carefully: `reviewPaper(int score, int paperID).` If every check below passes, the score is added to that paper with `addReviewScore`. A reviewer may review the same paper more than once; each call adds a score.
Throw Exceptions (check in exactly this order; stop at the first that applies):

#	Case	Throw
1	`score` < 1 or `score` > 10	`throw score;`
2	No paper with `paperID` is assigned to this reviewer	`throw paperID;`
3	That paper's status is not `Under_Review`	`throw paperID;`

Example: `reviewPaper(11, 2)` on a reviewer who does not have paper 2 throws `11`, not `2`, because the score is checked first.

6.3 `display` method

Prints the reviewer's name and a newline.

7 `Conference` Class

The conference stores pointers to papers and reviewers created elsewhere (the driver creates them with `new`). Both lists keep insertion order.

7.1 Adding and finding

- `addPaper(paper)`: appends the pointer, unless a paper with the same ID is already in the list (then nothing happens).
- `addReviewer(reviewer)`: appends the pointer. If a reviewer with the same name already exists, `throw invalid_argument(name);` and do not add.
- `findPaper(id)`: returns the pointer to the paper with that ID. If none, `throw id;`.
- `findReviewer(name)`: returns the pointer to the reviewer with that name. If none: `throw invalid_argument(name);`

7.2 `submitPaper` method

Finds the paper by ID (a missing ID throws the ID, as in `findPaper`). If its status is not `In_progress`, `throw paperID;`. Otherwise set its status to `Submitted`.

7.3 `assignPaperToReviewer` method

Assigns the paper to **the named reviewer only**, then sets the paper's status to `Under_Review`. Assigning a paper that is already `Under_Review` to another reviewer is allowed.
Throw Exceptions (check in exactly this order):

#	Case	Throw
1	No paper with `paperID`	`throw paperID;`
2	No reviewer named `reviewerName`	`throw invalid_argument(reviewerName);`
3	Status is neither `Submitted` nor `Under_Review`	`throw paperID;`
4	The reviewer is an author of the paper	`throw paperID;`

7.4 `decideAcceptance` method

For every paper whose status is `Under_Review` **and** that has at least one review score: if `averageScore()` $\geq$ 7.0, set it to `Accepted`; otherwise set it to `Rejected`. All other papers are left unchanged. This includes papers that are under review but have no score yet.

7.5 Printing

- `printAllReviewers()`: `display()` of each reviewer, in insertion order.
- `printAllPapers()`: `display()` of each paper, **newest first** (reverse insertion order).
- `display()`: `printAllReviewers()`, then `printAllPapers()`.

7.6 Worked example

Reviewers Rita and Sam are added. Papers 1 (GraphNets, Alice) and 2 (QuantumWalks, Bob) are created. Carol is added to paper 1. Both papers are submitted. Paper 1 is assigned to Rita and Sam, and paper 2 is assigned to Rita. Both papers are now `Under_Review`. Rita scores paper 1 with 8 and Sam scores it with 7, so the average is 7.5. Rita scores paper 2 with 5. `decideAcceptance()` then marks paper 1 `Accepted` and paper 2 `Rejected`. `display()` prints:

```
Rita
Sam
2 QuantumWalks: Bob 5.00 Rejected
1 GraphNets: Alice Carol 7.50 Accepted
```

8 Driver Program (given, not evaluated)

`local_tester/driver.h` contains last year's command-driven `main` as a function `runDriver`. It reads integer commands until it reads an unknown command number:

Cmd	Input	Action, then output on success
0	`0 <name>`	`addReviewer(new Reviewer(name))`; prints `0 OK`
1	`1 <title> <author>`	creates a `ConferencePaper`, `addPaper`, `display()`; prints `1 OK`
2	`2 <id> <author>`	`findPaper(id)->addAuthor(author)`, `display()`; prints `2 OK`
3	`3 <id>`	`submitPaper(id)`; prints `3 OK`
4	`4 <id> <reviewer>`	`assignPaperToReviewer(id, reviewer)`; prints `4 OK`
5	`5 <reviewer> <id> <score>`	`findReviewer(reviewer)->reviewPaper(score, id)`; prints `5 OK`
6	`6`	`decideAcceptance()`; prints `6 OK`
7	`7`	`display()`; prints `7 OK`
other	`<n>`	prints `<n> OK` and stops

If a command throws, the driver catches it and prints `<thrown int> <cmd> NOK`, or `<e.what()> <cmd> NOK` for an `invalid_argument`, then continues. See `Sample_Runs.pdf`.
To run it yourself (from the folder with `skeleton.cpp`):

```
cp skeleton.cpp local_tester/student.cpp
g++ -std=c++17 local_tester/run_driver.cpp -o drv
./drv < local_tester/sample1.in
```